Where Substitution Happens

Semantic substitution, deferred unquote and explicit substitution
in a paired-message rho calculus

L.G. Meredith
F1R3FLY.io

Working note, 1 October 2026

Abstract

The note *Unitary in the Equations, Measurement in the Reductions* [5] arrived, on the way to a quantum interpretation, at a small extension of the rho calculus that stands on its own: messages and names are pairs of a process and a datum drawn from a type D, dropping splits into a binary equation that preserves the datum and a unary drop resolved by semantic substitution, and replication is derived by reflection. Variables form their own category, given as a second type parameter V, not a sort of names. This note extracts that core, independently of any quantum reading, and uses it to compare three treatments of information exchange at interaction. In the first, the one adopted, the received code is installed by semantic substitution at the communication itself. In the second, substitution is syntactic and a separate unary rule $\&\,@\langle p,\, d\rangle \rightsquigarrow p$ unquotes later. In the third, the substitution is a process term left beside the continuation, in the style of explicit fusions. The two alternatives were proposed by Mike Stay. The argument against them has three parts: unary reductions that no interaction triggers do not fit the context-labelled bisimulation programme; both alternatives change the shape of bisimulation; and that change is the formal shadow of a practical cost, because once substitution is detached from communication the programmer must coordinate by hand which substitutions happen together and which happen in order. Worked examples show both burdens: on the programmer, and on the bisimulation proofs. A generic catalyst transformation, which turns any unary rule $A \rightsquigarrow B$ into $A \mid C \rightsquigarrow B$, answers the first objection; the note shows that it leaves the other two standing, and that it makes the remaining coordination burden explicit rather than removing it. The conclusion ties the argument to the rho combinators: the size of translations from languages with variables into combinator languages is usually read as evidence of the compression variables offer, but when delivery of values is concurrent part of that compression is a consolidation of coordination, and semantic substitution is what achieves it. Finally, a flat pattern binder $\mathsf{for}(@\langle X, \delta\rangle \leftarrow x)\, P$ meets all three objections with ordinary syntactic substitution, recovers the envelope datum that the core calculus erases, and contains the core calculus as its data-blind fragment.

Contents

1 Introduction **2**

2 The core calculus **3**

3 The two alternatives **5**
3.1 Deferred unquote 5
3.2 Explicit substitution 5

4 Where the unquote lives **6**

5 The argument **6**

6 The burden on the programmer **7**
6.1 Knowing that received code is live 7
6.2 Substitutions that must happen in order 8
6.3 Substitutions that must happen together 8
6.4 Names computed from received code 8

7 The burden on bisimulation **9**
7.1 Unary redexes and labels 9
7.2 The catalyst transformation 9
7.3 The relations get larger 11
7.4 Laws that fail outright 11
7.5 Substitutions in the state 11
7.6 Cost 12

8 What the alternatives offer **12**

9 Flat patterns **12**

10 Conclusion: two kinds of compression **14**

11 Open questions **15**

1 Introduction

Every process calculus must say what happens to the information carried by a message at the moment it is received. In the rho calculus [6] the question has an extra edge, because what is received is a quoted process, and the receiver may do two quite different things with it: treat it as a name, or run it. The original paper answers with *semantic substitution*: substitution is syntactic everywhere except at a drop $*y$, where it installs the received process in place. The quantum design session recorded in [5] rediscovered this choice under pressure, and in doing so produced a version of the calculus in which the choice is unusually easy to see. Names and messages are pairs, so there is a datum to lose; the binary drop keeps it and the unary drop erases it; and nothing else in the calculus touches the question.

That makes the core calculus a good place to compare alternatives. In conversation, Mike Stay proposed two. The first keeps substitution uniformly syntactic and moves execution of received code into a separate unary reduction. The second makes substitution a process term, after Wischik and Gardner's explicit fusions [3]. Both are natural, and both are standard moves elsewhere in the literature on substitution [1]. This note records why the paired-message calculus keeps semantic substitution, and makes the reasons concrete with examples.

Reading order. Section 2 states the core calculus. Section 3 states the two alternatives. Section 4 compares all three by asking where the unquote, and with it the erasure of the datum, takes place. Section 5 states the argument. Sections 6 and 7 give the examples, first the programmer's burden and then the burden on bisimulation proofs. Section 8 records what the alternatives do offer. Section 9 presents flat patterns, which resolve the comparison. Section 10 draws the conclusion, connecting the argument to the rho combinators, and Section 11 lists what remains open.

2 The core calculus

The construction is a type parameterisation of the rho calculus, by two parameters. Fix a nonempty set D of data, with a distinguished $d_0 \in D$, and a set V of variables. Nothing below uses any structure on D; in [5] it is the group $\mathbb{Z}/8$ of phase labels, but here it is arbitrary. We write v, w, y, z for elements of V.

Definition 2.1 (Syntax of $\mathsf{Sem}[D, V]$)**.**

$$\begin{array}{rcll} M & ::= & \langle P,\, d\rangle & \text{messages, } d \in D \\ x, u & ::= & @M & \text{names} \\ P & ::= & 0 \mid x!(M) \mid \mathsf{for}(v \leftarrow x)\, P \mid P \mid P \mid *(x,\, u) \mid *v & \text{processes, } v \in V \end{array}$$

The receipt $\mathsf{for}(v \leftarrow x)\, P$ binds v in P, and it is the only binder. There is no restriction operator and no persistent receipt: fresh names arise by quotation, and replication is derived (Proposition 2.8).

Variables and names are different syntactic categories. A variable occurs directly only under the unary drop $*v$. It reaches names through quotation, since a name quotes a process that may mention it, as in $@\langle *v,\, e\rangle$.

Remark 2.2 (Variables as a type parameter). In the original rho paper [6] variables and names share a syntactic category, for parsimony. That creates a kind of cognitive dissonance. Names in rho are never bound: they are quoted processes, compared structurally. Variables are placeholders, compared up to renaming. Techniques such as de Bruijn indices or levels belong to binders, and while they may be made to work for the names of the pi-calculus, they do not apply to the names of the rho calculus in general. Separating the categories, and making V a parameter, puts the binding discipline in one place and lets it be chosen independently of the name theory. With $V = \mathbb{N}$, read as de Bruijn indices or levels, α-conversion inside quotes disappears and name equality becomes syntactic identity modulo E1 and E2 below. A finite V controls the infinity that variables introduce: with $|V| = k$, a process can refer to at most k received values at once, inner receipts shadowing outer ones, in the manner of the finite-variable fragments of first-order logic. $V = \emptyset$ is the degenerate case with no receipts and no communication.

Definition 2.3 (Structural congruence)**.** $\equiv_S$ is the least congruence, closed under α-conversion of variables, satisfying

(E1) Monoid
$P \mid 0 \equiv_S P$, $\quad P \mid Q \equiv_S Q \mid P$, $\quad (P \mid Q) \mid R \equiv_S P \mid (Q \mid R)$.

(E2) Binary drop
$*(@\langle p,\, d\rangle,\, u) \equiv_S u!(\langle p,\, d\rangle)$.

Names are identified up to the congruence of what they quote: $@\langle p,\, d\rangle = @\langle q,\, e\rangle$ iff $p \equiv_S q$ and $d = e$.

Remark 2.4 (Why E2 is an equation). E2 could have been read as a notational abbreviation instead. It is folded into $\equiv_S$ for two reasons. As an equation it is a bijection between term shapes: every output $u!(M)$ is congruent to exactly one binary drop $*(@M,\, u)$, and nothing is lost in either direction. And because it sits in the quotient, the observation theory of Section 7, which is defined up to $\equiv_S$, never has to distinguish a forwarded message from its expansion. The binary drop forwards the whole pair named by its first argument, datum included.

Definition 2.5 (Semantic substitution)**.** Let $N = @\langle p,\, d\rangle$. The substitution $\{N/v\}$ commutes with every constructor, including quotation, renaming bound variables to avoid capture, and has a single non-trivial base case:

$$(*v)\{@\langle p,\, d\rangle/v\} \;=\; p, \qquad (*w)\{N/v\} \;=\; *w \quad (w \neq v).$$

Since v occurs directly only under the unary drop, semantic substitution replaces every $*v$, wherever it occurs, by p. Substitution acts under quotation, so a name built from received code changes at the moment the code arrives: $@\langle *v,\, e\rangle\{@\langle p,\, d\rangle/v\} = @\langle p,\, e\rangle$.

Definition 2.6 (Reduction)**.**

$$\begin{aligned}
&(\textsc{Comm}) && \mathsf{for}(v \leftarrow x)\, P \mid x!(\langle q,\, d\rangle) \;\rightsquigarrow\; P\{@\langle q,\, d\rangle/v\} \\
&(\textsc{Par}) && P \rightsquigarrow P' \;\Rightarrow\; P \mid Q \rightsquigarrow P' \mid Q \\
&(\textsc{Struct}) && P \equiv_S P',\; P' \rightsquigarrow Q',\; Q' \equiv_S Q \;\Rightarrow\; P \rightsquigarrow Q
\end{aligned}$$

COMM is the only rule that does any work, and every reduction is the meeting of a sender and a receiver.

Proposition 2.7 (Closure under reflection)**.** *Every communication substitutes a name of the form $@M$, and E2 takes such a name back to a message. The payload sort and the quoted sort coincide.*

Proposition 2.8 (Replication is derivable)**.** *Let $R_x := \mathsf{for}(v \leftarrow x)\,(x!(\langle *v,\, d_0\rangle) \mid *v)$ and, for P closed, $!P := x!(\langle R_x \mid P,\, d_0\rangle) \mid R_x$. Then*

$$!P \;\rightsquigarrow\; x!(\langle R_x \mid P,\, d_0\rangle) \mid R_x \mid P.$$

Proof. COMM substitutes $@\langle R_x \mid P,\, d_0\rangle$ for v. Both occurrences of $*v$ become $R_x \mid P$: the one inside the message re-emits the code for the next unfolding, and the one in process position installs a copy to run. Equivalently, by E2, $x!(\langle *v,\, d_0\rangle) \equiv_S *(@\langle *v,\, d_0\rangle,\, x)$, a binary drop of a name built from the received code. □

The proof shows two occurrences of the drop doing different jobs within a single step: one forwards a copy, the other installs a copy. Both happen at once, because both are effects of the same substitution. The forwarded copy carries the datum d_0 because the receiver writes d_0, which happens to be the datum it was sent with. That is no accident of the example, as the next proposition shows.

Proposition 2.9 (Envelope erasure)**.** *For all q, d, d' and every process P, the terms $S_d = x!(\langle q,\, d\rangle) \mid \mathsf{for}(v \leftarrow x)\, P$ and $S_{d'} = x!(\langle q,\, d'\rangle) \mid \mathsf{for}(v \leftarrow x)\, P$ have the same reduct:*

$$S_d \;\rightsquigarrow\; P\{@\langle q,\, d\rangle/v\} \;=\; P\{@\langle q,\, d'\rangle/v\} \quad \textit{and} \quad S_{d'} \;\rightsquigarrow\; P\{@\langle q,\, d'\rangle/v\}.$$

Proof. v occurs in P only under drops, directly or inside quotes, and semantic substitution sends each $*v$ to q whatever the datum. □

The datum attached to a message at the top level, which we call the *envelope* datum, is erased at the COMM, and no receiver can record it. Every name a receiver builds from received code takes its datum from the receiver: $r!(\langle *v,\, e\rangle)$, $*(@\langle *v,\, e\rangle,\, r)$ and $@\langle *v,\, e\rangle!(\langle 0,\, e\rangle)$ all record e, never d. Data in the *payload* survive, because the payload process survives: if the sender ships $\langle k!(\langle 0,\, d\rangle),\, e\rangle$, then $@\langle *v,\, e'\rangle$ resolves to $@\langle k!(\langle 0,\, d\rangle),\, e'\rangle$, which records d. So a receiver cannot recover the envelope datum, but a cooperating sender can always make a datum recoverable by moving it into the payload. Section 9 gives a receiver-side alternative.

Remark 2.10 (Installation and execution). Semantic substitution fixes the moment received code is *installed*: at the COMM. It does not force the code to run then. If $*v$ is guarded, say $\mathsf{for}(v \leftarrow x)\,\mathsf{for}(z \leftarrow c)\, *v$, then p is installed under the guard at the first COMM and runs when the guard is lifted by the second. Both moments are communications, and both are under the program's control.

3 The two alternatives

3.1 Deferred unquote

Definition 3.1 ($\mathsf{Unq}[D, V]$)**.** Replace the unary drop with an unquote operator, applicable to variables and to names, and substitute syntactically:

$$\begin{array}{rcl} P & ::= & 0 \mid x!(M) \mid \mathsf{for}(v \leftarrow x)\, P \mid P \mid P \mid *(x,\, u) \mid \&\, v \mid \&\, x \\ (\textsc{Comm}_{\textsc{U}}) & & \mathsf{for}(v \leftarrow x)\, P \mid x!(\langle q,\, d\rangle) \;\rightsquigarrow\; P\{@\langle q,\, d\rangle/v\}_{\mathrm{syn}} \\ (\textsc{Unq}) & & \&\, @\langle p,\, d\rangle \;\rightsquigarrow\; p \end{array}$$

with E1, E2, PAR and STRUCT as before. The syntactic substitution $\{N/v\}_{\mathrm{syn}}$ replaces $\&\, v$ by $\&\, N$, everywhere including under quotation, and does nothing else.

The operator must apply to names as well as variables, because syntactic substitution turns $\&\, v$ into $\&\, @\langle q,\, d\rangle$. A consequence is that $\&\, @\langle p,\, d\rangle$ can be written directly in source text, as a term that reduces with nobody else involved. In $\mathsf{Sem}[D, V]$ the corresponding term is never formed. Another consequence runs opposite to Proposition 2.9: in $\mathsf{Unq}[D, V]$ the envelope datum is never erased from names, since $@\langle \&\, v,\, e\rangle$ becomes $@\langle \&\, @\langle q,\, d\rangle,\, e\rangle$, which records d whether or not anyone wants it to.

3.2 Explicit substitution

The second proposal leaves the substitution itself in the term, more or less following explicit fusions [3, 7]. The rules below are a reconstruction, made to have something definite to argue about; the precise rules intended may differ, and the argument is written so that it does not depend on the details.

Definition 3.2 ($\mathsf{ES}[D, V]$)**.** Extend the syntax of $\mathsf{Unq}[D, V]$ with a substitution term and a scope for the variable it governs:

$$\begin{array}{rcl} P & ::= & \cdots \mid [\, N/v\,] \mid (v)\, P \\ (\textsc{Comm}_{\textsc{E}}) & & \mathsf{for}(v \leftarrow x)\, P \mid x!(M) \;\rightsquigarrow\; (v)\, (P \mid [\, @M/v\,]) \end{array}$$

with UNQ for unquoting, and the fusion-style equations

(X1) Propagation
$(v)\, (P \mid [\, N/v\,]) \equiv_S (v)\, (P\{N/v\}_{\mathrm{syn}} \mid [\, N/v\,])$.

(X2) Collection
$(v)\, [\, N/v\,] \equiv_S 0$.

(X3) Extrusion
$((v)\, P) \mid Q \equiv_S (v)\, (P \mid Q)$ when $v \notin \mathrm{fv}(Q)$.

The scope is needed. In the receipt, v is bound by the input prefix; $\mathrm{COMM}_{\mathsf{E}}$ removes the prefix, so something else must keep v from becoming a free, global variable. In the fusion calculus that job is done by restriction, which keeps fusions local. The rho calculus has no restriction, deliberately, since fresh names arise by quotation, so $\mathsf{ES}[D, V]$ must import one.

There are two ways to read the propagation of the substitution term, and they lead to different calculi.

- **Equational reading.** X1 holds as stated. Then by X1, X3 and X2, $(v)\, (P \mid [\, N/v\,]) \equiv_S P\{N/v\}_{\mathrm{syn}}$, so $\mathsf{ES}[D, V]$ coincides with $\mathsf{Unq}[D, V]$ up to $\equiv_S$ and adds only the bookkeeping of the scope. Everything said about $\mathsf{Unq}[D, V]$ below then applies to it unchanged.

- **Interactional reading.** X1 is dropped, and each unguarded occurrence of $\&\, v$ is resolved by its own reduction against the substitution term, which persists until collected: $[\,@\langle p,\, d\rangle/v\,] \mid \&\, v \rightsquigarrow [\,@\langle p,\, d\rangle/v\,] \mid p$. Each resolution is now an interaction, which answers the first objection below, but a single COMM is followed by one resolution step per occurrence, and occurrences under quotation are never reached, because nothing reduces inside a name.

4 Where the unquote lives

The three calculi agree on everything except the treatment of the unary drop. The binary drop is the same equation in all of them. Without the unary drop, nothing runs received code, recursion is lost, and with linear receipts every term strongly normalises [5]. So the unary drop is exactly where recursion comes from, and the whole comparison concerns one operator.

The unary drop is also the one place the datum is erased: $@\langle p,\, d\rangle \mapsto p$ is many-to-one. It is therefore useful to ask, of each design, which step performs that erasure. A fourth placement, the equation $*@\langle p,\, d\rangle \equiv_S p$, completes the picture.

Placement	**Triggered by interaction**	**Atomic with the COMM**	**Where d is erased**
Equation, $*@\langle p,\, d\rangle \equiv_S p$	no step	no step	in $\equiv_S$, at no cost
Semantic substitution ($\mathsf{Sem}[D, V]$)	yes	yes	at the COMM
Deferred unquote ($\mathsf{Unq}[D, V]$)	no	no	at a UNQ step, per occurrence
Deferred unquote, catalysed ($\mathsf{Unq}[D, V]^C$)	yes, with the catalyst	no	at a catalysed step, per occurrence
Explicit substitution ($\mathsf{ES}[D, V]$), equational	no	no	as in $\mathsf{Unq}[D, V]$
Explicit substitution ($\mathsf{ES}[D, V]$), interactional	yes	no	at each resolution
Flat patterns ($\mathsf{Pat}[D, V]$)	yes	yes	at the COMM, only for components the receiver discards

The equational placement is ruled out in [5] because equations are free while information erasure is not: an equation that forgets d makes $\equiv_S$ identify configurations that differ in data, and cost accounting has nowhere to charge for the loss. Semantic substitution charges the erasure to the COMM, which is already the step that interaction pays for. The deferred unquote charges it to a step that no interaction pays for, unless it is catalysed (Section 7.2), in which case the catalyst pays. Explicit substitution either inherits that placement or spreads the erasure across as many steps as there are occurrences.

5 The argument

1. **Reductions should be interactions.** A unary rule like $\&\, @\langle p,\, d\rangle \rightsquigarrow p$, which fires without any interaction triggering it, does not fit the context-labelled bisimulation programme [4]. There, a transition is labelled by the minimal context that completes a redex, and every redex of $\mathsf{Sem}[D, V]$ is a sender and receiver meeting. A redex that is complete on its own contributes transitions that no context takes part in, and that the environment can

neither cause nor see except through their effects. This objection, unlike the other two, has a generic answer, the catalyst transformation of Section 7.2.

2. **Both proposals change bisimulation.** $\mathsf{Unq}[D, V]$ adds internal steps between a communication and the running of what it delivered. $\mathsf{ES}[D, V]$ puts substitution terms into the states being compared. Either way the equivalence, and the proofs about it, must be rethought.

3. **That change reflects a cost to the programmer.** In $\mathsf{Sem}[D, V]$ the substitution happens atomically, at the COMM, and the programmer coordinates when and where substitutions happen with the full power of the language, because every substitution is a communication. In the alternatives, substitutions are detached from the communications that produced them, and the programmer has to do extra work to make sure that substitutions that must coincide do, and substitutions that must be ordered are.

Points 2 and 3 are one point seen from two sides: the more complicated bisimulation is the formal record of the coordination the programmer now has to do. The next two sections give examples of each side.

6 The burden on the programmer

Throughout, a, b, c, x, x_1, x_2 and ack are fixed, distinct quoted names, and $e \in D$ is an arbitrary datum. Write $P{\downarrow_a}$ when $P \equiv_S a!(M) \mid Q$ for some M and Q, so that P offers an output on a.

6.1 Knowing that received code is live

Example 6.1 (Install, then acknowledge)**.** A receiver installs code it is sent and then acknowledges:

$$S \;=\; x!(\langle p,\, d\rangle) \mid \mathsf{for}(y \leftarrow x)\,(*y \mid ack!(\langle 0,\, e\rangle)).$$

In $\mathsf{Sem}[D, V]$, $S \rightsquigarrow p \mid ack!(\langle 0,\, e\rangle)$. Whoever consumes the acknowledgement does so in a state where p is already installed: the acknowledgement means what it says. In $\mathsf{Unq}[D, V]$,

$$S \;\rightsquigarrow\; \&\,@\langle p,\, d\rangle \mid ack!(\langle 0,\, e\rangle),$$

and the acknowledgement can be consumed, and acted on, before UNQ fires.

The natural repair is for the receiver to acknowledge *after* the unquote. That cannot be done from the receiver's side. Suppose the receiver wraps the code, sends the wrapper to itself, and unquotes it:

$$\mathsf{for}(y \leftarrow x)\,\big(w!(\langle \&\, y \mid ack!(\langle 0,\, e\rangle),\, e\rangle) \mid \mathsf{for}(z \leftarrow w)\;\&\, z\big).$$

After both communications and the UNQ of the wrapper, the state is $\&\,@\langle p,\, d\rangle \mid ack!(\langle 0,\, e\rangle)$, which is where it started. Every wrapper leaves the inner unquote pending.

Proposition 6.2 (No receiver-side signal of liveness)**.** *In $\mathsf{Unq}[D, V]$, no part of the receiver can be causally downstream of the step $\&\,@\langle p,\, d\rangle \rightsquigarrow p$, except through actions of p itself.*

Proof sketch. UNQ has a single residual, p, and no partner. Anything else in the term is unchanged by the step and so cannot depend on it. The only events that follow it causally are those of p. □

So in $\mathsf{Unq}[D, V]$ the acknowledgement must be built into the shipped code: the *sender* must send $\langle p \mid ack!(\langle 0,\, e\rangle),\, d\rangle$. That moves a receiver's coordination problem across the trust boundary to the sender, and it changes the message, hence the name $@M$, hence any protocol keyed on that name. In $\mathsf{Sem}[D, V]$ the receiver needs no help. Its continuation running is the evidence that the code is installed, because both are the same step.

6.2 Substitutions that must happen in order

Example 6.3 (Activation follows reception)**.** A receiver takes two pieces of code in sequence and runs each:

$$T \;=\; x!(\langle p_1,\, d_1\rangle) \mid x!(\langle p_2,\, d_2\rangle) \mid \mathsf{for}(y_1 \leftarrow x)\,\big(*y_1 \mid \mathsf{for}(y_2 \leftarrow x)\,*y_2\big).$$

Taking p_1 as the first one received, in $\mathsf{Sem}[D, V]$ every run passes through $p_1 \mid \mathsf{for}(y_2 \leftarrow x)\,*y_2 \mid x!(\langle p_2,\, d_2\rangle)$: the first code is installed before the second can be received. In $\mathsf{Unq}[D, V]$ this run is available:

$$\begin{aligned} T \;&\rightsquigarrow\; \&\,@\langle p_1,\, d_1\rangle \mid \mathsf{for}(y_2 \leftarrow x)\;\&\,y_2 \mid x!(\langle p_2,\, d_2\rangle) \\ &\rightsquigarrow\; \&\,@\langle p_1,\, d_1\rangle \mid \&\,@\langle p_2,\, d_2\rangle \\ &\rightsquigarrow\; \&\,@\langle p_1,\, d_1\rangle \mid p_2 \\ &\rightsquigarrow\; p_1 \mid p_2. \end{aligned}$$

The second code is live while the first is not.

The order of reception no longer fixes the order of activation. When that order matters, as when p_2 is an update meant to supersede p_1, the programmer must restore it, and by Proposition 6.2 the only way is the one from Example 6.1: the sender of p_1 builds in a signal, and the receiver guards the second receipt on it.

6.3 Substitutions that must happen together

Example 6.4 (Simultaneous availability)**.** Two pieces of code arrive on different channels, from different senders, and should become available together. Let $p_1 = a!(\langle 0,\, e\rangle)$ and $p_2 = b!(\langle 0,\, e\rangle)$, and

$$U \;=\; x_1!(\langle p_1,\, d_1\rangle) \mid x_2!(\langle p_2,\, d_2\rangle) \mid \mathsf{for}(y_1 \leftarrow x_1)\,\mathsf{for}(y_2 \leftarrow x_2)\,(*y_1 \mid *y_2).$$

In $\mathsf{Sem}[D, V]$, p_1 is installed under the guard on x_2 at the first COMM, and both are released by the second: $U \rightsquigarrow\rightsquigarrow p_1 \mid p_2$. Along every run, a state offers output on a only if it also offers output on b: $P{\downarrow_a}$ implies $P{\downarrow_b}$. In $\mathsf{Unq}[D, V]$, after both COMMs and one UNQ the state is $p_1 \mid \&\,@\langle p_2,\, d_2\rangle$, with $\downarrow_a$ but $\not\downarrow_b$.

As in the previous examples, the receiver cannot repair this. Packaging the two received codes into one wrapper, $w!(\langle \&\,y_1 \mid \&\,y_2,\, e\rangle)$, and unquoting it leaves $\&\,@\langle p_1,\, d_1\rangle \mid \&\,@\langle p_2,\, d_2\rangle$, two independent pending steps again. In $\mathsf{Unq}[D, V]$, two codes received from independent senders cannot be made live simultaneously by the receiver alone. In $\mathsf{Sem}[D, V]$ it takes no effort at all, because a single COMM installs every drop of the variables it has bound so far.

With k unguarded drops of received variables, a single COMM of $\mathsf{Sem}[D, V]$ corresponds in $\mathsf{Unq}[D, V]$ to a COMM followed by k independent unquotes, with 2^k reachable states of partial activation. Each of these is a state the programmer must either show to be harmless or rule out.

6.4 Names computed from received code

Because substitution in $\mathsf{Sem}[D, V]$ acts under quotation, a name built from received code is resolved at the COMM along with everything else. This is how rho programs compare code: two processes rendezvous on a name exactly when the code they quote is congruent.

Example 6.5 (Naming the code that arrived)**.** Let $n(y) = @\langle *y,\, d_0\rangle$, a name computed from received code, and

$$V \;=\; x!(\langle p,\, d\rangle) \mid \mathsf{for}(y \leftarrow x)\, n(y)!(\langle 0,\, e\rangle) \mid \mathsf{for}(z \leftarrow @\langle p,\, d_0\rangle)\, Q.$$

In $\mathsf{Sem}[D, V]$, COMM gives $n(y)\{@\langle p,\, d\rangle/y\} = @\langle p,\, d_0\rangle$, so the output meets the third component and Q runs. In $\mathsf{Unq}[D, V]$ the output lands on $@\langle \& @\langle p,\, d\rangle,\, d_0\rangle$, which is a different name from $@\langle p,\, d_0\rangle$, because UNQ is a reduction, not an equation, and nothing reduces inside a quote. The rendezvous never happens.

The workaround in $\mathsf{Unq}[D, V]$ is to listen on $@\langle \& @\langle p,\, d\rangle,\, d_0\rangle$. But that name depends on the datum d of the message in which the code happened to travel. The same code obtained by three routes, received with datum d, received with datum d', or written literally, gets three different names. Code identity becomes route-dependent, and the programmer must track the route of every piece of code whose name is used.

The same problem breaks a standard rho idiom, replying on a name derived from the request. With a reply datum e_r agreed in advance,

$$x!(\langle q,\, e\rangle) \mid \mathsf{for}(r \leftarrow @\langle q,\, e_r\rangle)\, S \;\mid\; \mathsf{for}(y \leftarrow x)\, @\langle *y,\, e_r\rangle!(\langle q',\, e\rangle).$$

In $\mathsf{Sem}[D, V]$ the server's reply is on $@\langle q,\, e_r\rangle$ as soon as the request is received, and it meets the client. In $\mathsf{Unq}[D, V]$, and in $\mathsf{ES}[D, V]$ under the equational reading, it lands on $@\langle \& @\langle q,\, e\rangle,\, e_r\rangle$ and never meets the client. Under the interactional reading of $\mathsf{ES}[D, V]$ it is worse: the occurrence of y is under quotation, no resolution step reaches it, and the reply channel keeps a free variable for good. In $\mathsf{Sem}[D, V]$ the reply datum must be agreed in advance, because the envelope datum e is erased (Proposition 2.9); Section 9 removes that restriction, letting the server reply on exactly the name $@\langle q,\, e\rangle$.

7 The burden on bisimulation

7.1 Unary redexes and labels

In the approach of Leifer and Milner [4], an agent a has a transition $a \xrightarrow{C} a'$ when $C[a] \rightsquigarrow a'$ and C is a minimal context that makes this happen; their congruence theorem is for the strong equivalences built on these labels. In $\mathsf{Sem}[D, V]$ every redex consists of a receiver and a sender, so a label is either the missing half of a communication or the identity, when both halves are already present in the agent. In $\mathsf{Unq}[D, V]$ the term $\& @\langle p,\, d\rangle$ is a complete redex on its own. It contributes identity-labelled transitions that correspond to no communication at all.

Strong equivalences count these steps. $\& @\langle p,\, d\rangle$ has exactly one transition and its target is p, so it is not strongly bisimilar to p except in degenerate cases. Every law of $\mathsf{Sem}[D, V]$ that involves running received code therefore survives in $\mathsf{Unq}[D, V]$ at best as a weak law, and extending the context-labelled congruence results from strong to weak equivalences requires further conditions that must then be checked.

7.2 The catalyst transformation

Mike Stay pointed out that the first objection has a generic answer. It comes from a device the author introduced in 2014 to repair a categorical semantics for the pi-calculus, published as the catalysis construction of [9], where it restricts the 2-morphisms interpreting rewrites to the contexts in which they may apply. The idea is to make a unary reduction depend on a *catalyst*, a fixed process whose presence enables it.

Definition 7.1 (Catalyst transformation)**.** Fix a process C. Every rule of the form $A \rightsquigarrow B$ in which no interaction triggers the step is replaced by

$$A \mid C \;\rightsquigarrow\; B.$$

In the reactive system, the step then has the labelling context $[\,] \mid C$. In the *catalytic* variant the catalyst is returned, $A \mid C \rightsquigarrow B \mid C$.

Applied to $\mathsf{Unq}[D, V]$, this gives a calculus $\mathsf{Unq}[D, V]^C$ with

$$(\textsc{Unq}_C) \qquad \& \, @\langle p,\, d\rangle \mid C \;\rightsquigarrow\; p \qquad \text{or} \qquad \& \, @\langle p,\, d\rangle \mid C \;\rightsquigarrow\; p \mid C.$$

Now $\& \, @\langle p,\, d\rangle$ is no longer a complete redex on its own. Its transition is

$$\& \, @\langle p,\, d\rangle \xrightarrow{[]|C} p,$$

labelled by the context that supplies the catalyst, and every reduction of $\mathsf{Unq}[D, V]^C$ is an interaction. The first objection is met in full, and generically: any calculus with unary rules can be brought inside the context-labelled programme this way.

The interactional reading of $\mathsf{ES}[D, V]$ is an instance of the catalytic variant, with the substitution term itself as the catalyst: $[\,N/y\,] \mid \& \, y \rightsquigarrow [\,N/y\,] \mid p$. This explains why that reading already answered the first objection in Section 3.

The transformation makes each unquote an interaction. It does not make the unquote part of the communication that delivered the code, and the remaining objections are about exactly that.

- **Atomicity is not restored.** A COMM followed by k unguarded unquotes is still $k + 1$ steps, now k of them catalysed. The 2^k partial states of Example 6.4 and of the next subsection remain, and so does the simultaneity problem: one catalysed step enables one unquote.
- **Quotation is untouched.** No reduction acts inside a quote, catalysed or not, so Example 6.5 and Proposition 7.2 hold for $\mathsf{Unq}[D, V]^C$ as stated.
- **Strong equivalences still separate $\& \, @\langle p,\, d\rangle$ from p.** The step is now labelled $[\,] \mid C$ instead of the identity, but it is still a step, so strong context-labelled bisimilarity still distinguishes the pending unquote from the running code.

What changes is who can control the timing. If C is ambient, for instance a catalyst that is always present and always returned, the labels change but the programmer gains no handle on when unquotes fire. If C is something the program emits and withholds, say an output $\kappa!(\langle 0,\, d_0\rangle)$ on a name κ that the program holds, then the program can decide when each unquote is allowed to happen. That helps with ordering, but only partly. To activate p_2 only after p_1 in Example 6.3, the receiver needs evidence that the first unquote has completed. Under $A \mid C \rightsquigarrow B$ there is none: the catalyst is consumed and nothing new appears. Under the catalytic variant the catalyst reappears, but its presence does not distinguish "not yet used" from "used and returned", and a receiver that waits on it competes with the unquote for it. So Proposition 6.2 carries over. Evidence of completion exists only if the catalysed rule is designed to emit a signal, $A \mid C \rightsquigarrow B \mid C'$.

A catalyst that signals completion is, however, a small synchronisation protocol laid alongside every unquote: release the catalyst, wait for the signal, proceed. That is the coordination of the third objection, written out explicitly. The catalyst transformation therefore does not remove the programmer's burden; it makes the burden visible as protocol. It shows that what separates the alternatives from semantic substitution is not the absence of an interaction but the absence of atomicity. $\mathsf{Sem}[D, V]$ needs no catalyst, because its only unquote is already part of the one interaction that delivered the code.

7.3 The relations get larger

Consider a receiver with k unguarded drops of the received variable,

$$W \;=\; x!(\langle p,\, d\rangle) \mid \mathsf{for}(y \leftarrow x)\,(*y \mid \cdots \mid *y).$$

In $\mathsf{Sem}[D,V]$, W has one reduction, to $p^{|k}$, and any bisimulation containing W needs one more pair to account for it. In $\mathsf{Unq}[D,V]$, W reduces to $(\&\,@\langle p,\, d\rangle)^{|k}$, and then through every subset of the k unquotes. A weak bisimulation relating W to its $\mathsf{Sem}[D,V]$ counterpart must contain a pair for each of the 2^k partial states, each related to $p^{|k}$ or to an intermediate of p's own behaviour. These are the same 2^k states that burden the programmer in Example 6.4, now appearing as proof obligations. Up-to techniques can compress them, but they have to be developed and justified, whereas in $\mathsf{Sem}[D,V]$ they are not needed.

7.4 Laws that fail outright

The extra steps of $\mathsf{Unq}[D,V]$ can be absorbed by moving to weak equivalence. The name problem of Example 6.5 cannot.

Proposition 7.2. *Let*

$$S_1 = x!(\langle p,\, d\rangle) \mid \mathsf{for}(y \leftarrow x)\,@\langle *y,\, d_0\rangle!(\langle 0,\, e\rangle),$$

$$S_2 = x!(\langle p,\, d\rangle) \mid \mathsf{for}(y \leftarrow x)\,@\langle p,\, d_0\rangle!(\langle 0,\, e\rangle),$$

with $y \notin \mathrm{fv}(p)$, and $*y$ read as $\&\,y$ in $\mathsf{Unq}[D,V]$. Then S_1 and S_2 are strongly barbed bisimilar in $\mathsf{Sem}[D,V]$, and not even weakly barbed bisimilar in $\mathsf{Unq}[D,V]$.

Proof sketch. In $\mathsf{Sem}[D,V]$ both have the barb $\downarrow_x$ and a single reduction, to the same term $@\langle p,\, d_0\rangle!(\langle 0,\, e\rangle)$. In $\mathsf{Unq}[D,V]$ the reducts are $@\langle \&\,@\langle p,\, d\rangle,\, d_0\rangle!(\langle 0,\, e\rangle)$ and $@\langle p,\, d_0\rangle!(\langle 0,\, e\rangle)$. Both are stuck, and their barbs are on distinct names, since $\&\,@\langle p,\, d\rangle \not\equiv_S p$. No sequence of internal steps reconciles them, because no step acts inside a quote. □

The bisimilarity in $\mathsf{Sem}[D,V]$ is not a congruence: a context can send different code on x. The point is that in $\mathsf{Unq}[D,V]$ the two terms are distinguished even without a context. The law "receiving code and naming it is the same as naming the code" holds in $\mathsf{Sem}[D,V]$ and fails in $\mathsf{Unq}[D,V]$, however weak the equivalence. Bisimulation proofs in $\mathsf{Unq}[D,V]$ must keep track of the route by which each named piece of code arrived, which is Example 6.5 restated as a proof obligation.

7.5 Substitutions in the state

Under the equational reading, $\mathsf{ES}[D,V]$ reduces to $\mathsf{Unq}[D,V]$ up to $\equiv_S$, so bisimulation up to $\equiv_S$ absorbs the substitution terms and the previous subsections apply unchanged. Under the interactional reading the substitution terms are genuine components of the state. A bisimulation must relate $(y)\,(P \mid [N/y])$, and each of its partial resolutions, to $P\{N/y\}$, and the number of internal steps a single COMM gives rise to depends on the number of occurrences of y, which is a syntactic accident. Names with a free variable, such as $@\langle \&\,y,\, d_0\rangle$, mean different channels under different surrounding substitutions, so barbs must be read relative to their scope. This is why the fusion calculus works with hyperequivalence, bisimulation closed under substitutions [7]. That is a heavier notion than anything $\mathsf{Sem}[D,V]$ needs, since in $\mathsf{Sem}[D,V]$ no state ever contains an unresolved received name.

7.6 Cost

In cost-accounted rho, cost is charged to reductions. In $\mathsf{Sem}[D, V]$ the cost of a run is a function of its communications. In $\mathsf{Unq}[D, V]$ it also depends on how many drops the receiver happened to write, and under the interactional reading of $\mathsf{ES}[D, V]$ on how many occurrences of the variable there are in any position. A cost-aware equivalence therefore distinguishes programs that differ only in how often they mention what they received. This is the cost-accounting version of the remark in Section 4: the alternatives charge for erasure at steps that interaction does not pay for.

8 What the alternatives offer

The alternatives have real merits, and the argument does not depend on denying them.

- **Provenance.** In $\mathsf{Unq}[D, V]$ a name built from received code records the envelope datum of the message the code arrived in, whether or not anyone wants it to. In $\mathsf{Sem}[D, V]$ that datum is erased at the COMM (Proposition 2.9), and it is available downstream only if the sender moves it into the payload. Neither calculus leaves the choice to the receiver; the flat patterns of Section 9 do.
- **Sharing.** Explicit substitution earns its place in implementations, where it avoids copying large terms and lets a substitution be shared [1]. An implementation of $\mathsf{Sem}[D, V]$ may well use explicit substitutions internally. The argument here is about the calculus, which fixes what is observable: whatever the machinery, the installation of received code must look atomic with the communication that delivered it.
- **Uniform substitution.** $\mathsf{Unq}[D, V]$ has a simpler substitution operation, which is pleasant metatheoretically. The price is paid elsewhere: in an extra reduction rule, in weak equivalences, and in the failure of Proposition 7.2. Section 9 shows that uniform substitution can be had without that price.

9 Flat patterns

The discussion so far leaves two loose ends. Semantic substitution is atomic and interaction-triggered, but its substitution has a special case and it erases the envelope datum. The alternatives make substitution uniform, but only by detaching it from the communication. A pattern in the binder of the receipt resolves both. Of the designs considered here it is the most conservative.

Definition 9.1 ($\mathsf{Pat}[D, V]$)**.** Let the variables come in two sorts, $V = V_P \uplus V_D$: process variables $X, Y \in V_P$ and data variables $\delta \in V_D$.

$$
\begin{array}{rcll}
t & ::= & d \mid \delta & \text{data terms, } d \in D \\
M & ::= & \langle P,\, t \rangle & \text{messages} \\
x, u & ::= & @M & \text{names} \\
P & ::= & 0 \mid x!(M) \mid \mathsf{for}(@\langle X, \delta \rangle \leftarrow x)\, P \mid P \mid P \mid *(x,\, u) \mid X & \text{processes}
\end{array}
$$

The receipt binds X and δ in P. Structural congruence is as in Definition 2.3, with E2 read for names $@\langle p,\, t \rangle$. Reduction has PAR and STRUCT as before, and

$$
(\textsc{Comm}_{\mathrm{P}}) \qquad \mathsf{for}(@\langle X, \delta \rangle \leftarrow x)\, P \mid x!(\langle q,\, d \rangle) \;\rightsquigarrow\; P\{q/X\}_{\mathrm{syn}}\{d/\delta\}_{\mathrm{syn}}
$$

where the substitution is ordinary, capture-avoiding and syntactic, and acts under quotation.

The unary drop has gone. A process variable stands for the received process itself, so it occupies process positions directly, and substituting for it is plain syntactic substitution, as in the higher-order pi-calculus. The special case of semantic substitution, $(*v)\{@\langle p,\, d\rangle/v\} = p$, has been absorbed into the binder: the pattern takes the quote apart at the moment of receipt, so there is nothing left to unquote.

Proposition 9.2 ($\mathsf{Sem}[D,V]$ is the data-blind fragment of $\mathsf{Pat}[D,V]$)**.** *Define* $[\![\cdot]\!]$ *from* $\mathsf{Sem}[D,V]$ *to* $\mathsf{Pat}[D,V]$ *homomorphically, including under quotation, except that*

$$[\![\mathsf{for}(v \leftarrow x)\, P]\!] = \mathsf{for}(@\langle X_v, \delta_v\rangle \leftarrow [\![x]\!])\, [\![P]\!], \qquad [\![*v]\!] = X_v,$$

with X_v *and* δ_v *fresh. Then* $[\![P\{@\langle q,\, d\rangle/v\}]\!] = [\![P]\!]\{[\![q]\!]/X_v\}_{\mathrm{syn}}\{d/\delta_v\}_{\mathrm{syn}}$*, and the translation is a strong operational correspondence:* $P \rightsquigarrow P'$ *implies* $[\![P]\!] \rightsquigarrow [\![P']\!]$*, and* $[\![P]\!] \rightsquigarrow Q$ *implies* $Q \equiv_S [\![P']\!]$ *for some* $P \rightsquigarrow P'$*.*

Proof sketch. The substitution identity is by induction on P. The only base case is $*v$, sent to q on the left and X_v to $[\![q]\!]$ on the right; the substitution for δ_v is vacuous, since δ_v does not occur in $[\![P]\!]$. A flat pattern always matches, so COMM and $\mathrm{COMM}_{\mathsf{P}}$ fire on exactly the same redexes, and the identity carries each reduct to the other. □

So $\mathsf{Sem}[D,V]$ is $\mathsf{Pat}[D,V]$ with every data variable left unused, which explains Proposition 2.9: the core calculus erases the envelope datum because it has no way to bind it. With the binder in hand:

- **All three objections are met.** Each communication is one step, triggered by an interaction, and installs every occurrence at once. There is no unquote rule, no catalyst, and no deferred work.
- **Substitution is uniformly syntactic.** This was the motivation for the deferred unquote, and $\mathsf{Pat}[D,V]$ has it without paying for it in atomicity.
- **The envelope datum is the receiver's to keep.** The process $\mathsf{for}(@\langle X, \delta\rangle \leftarrow x)\, r!(\langle 0,\, \delta\rangle)$ records it. Erasure still happens at the COMM, but only for components the receiver chooses not to use.
- **The received name can be rebuilt exactly.** $@\langle X,\, \delta\rangle$ resolves to $@\langle q,\, d\rangle$, the sender's own name. Names built from received code coincide with names built from literal code, so Proposition 7.2 has no analogue, and the server in the reply idiom of Section 6 can reply on the request's own name, with no reply datum agreed in advance.
- **Replication forwards the sender's datum.** With $R_x := \mathsf{for}(@\langle X, \delta\rangle \leftarrow x)\, (x!(\langle X,\, \delta\rangle) \mid X)$, the forwarded message is exactly the one received, and by E2 $x!(\langle X,\, \delta\rangle) \equiv_S *(@\langle X,\, \delta\rangle,\, x)$ is a binary drop of the received name itself.

Projections as catalysts. A variant closer to the alternatives makes data terms of the calculus and adds two projections as catalysts, $*@\langle p,\, d\rangle \mid \pi_1 \rightsquigarrow p$ and $*@\langle p,\, d\rangle \mid \pi_2 \rightsquigarrow d$, with syntactic substitution. It also recovers the envelope datum, and by the catalyst transformation every step is an interaction. But it inherits the loss of atomicity: a receiver that wants both components pays a COMM and two projections, and needs two copies of the drop, since each is consumed. It also adds a new burden. A catalyst is not addressed to any particular drop, so in $*@\langle p_1,\, d_1\rangle \mid \pi_1 \mid *@\langle p_2,\, d_2\rangle \mid \pi_2$ the intended $p_1 \mid d_2$ and the crossed $d_1 \mid p_2$ are equally reachable. Preventing the crossing requires guards or name-indexed projections, which reintroduce the channels the catalysts stood in for. The pattern binder addresses both components through the binder, so there is nothing to cross.

Why flat. The conservativity depends on the pattern being flat. Deep patterns such as $\mathsf{for}(@\langle \mathsf{for}(z \leftarrow w)\, R, \delta\rangle \leftarrow x)\, P$ require matching modulo $\equiv_S$, and since parallel composition is a commutative monoid that is associative-commutative matching, NP-hard in general. They also turn the binder into something close to a query in a spatial logic. The flat pattern $@\langle X, \delta\rangle$ is linear and always matches, so it costs no more than the receipt it replaces.

10 Conclusion: two kinds of compression

Translating a term language with variables and nested scopes into a combinator language, which is much flatter, typically blows up the size of terms. The effect is especially striking for Yoshida's concurrent combinators for the pi-calculus [10]. On the family $P_d = \mathsf{for}(x_1 \leftarrow a)\, \mathsf{for}(x_2 \leftarrow a) \cdots \mathsf{for}(x_d \leftarrow a)\, 0$, of size $d+1$, her prefix elimination produces $(3^{d-1}+1)/2$ atoms [8]. It is natural to take such blow-ups as strong evidence for the compression that variables provide.

The usual account of that compression is abstraction. A variable is a single point of reference, and a binder consolidates the mechanism that carries a value from the environment to its place in the term. SKI has no such mechanism [2]. Bracket abstraction rebuilds the route from the outside of a term to each use of a value on the inside, again and again: at every application node, and once more for every level of nesting. Measured against SKI, the value of variables appears to be the consolidation of these *routes*.

There is a richer perspective, and this note is what exposes it. When the calculations that carry a value to its uses are allowed to be concurrent, there is a second cost beyond routing: *coordination*. Delivering a value to several occurrences becomes several independent events. Someone must then arrange that deliveries which have to coincide do, and deliveries which have to be ordered are. Sections 6 and 7 show that this cost is real, both for the programmer and in the proofs, and the catalyst transformation of Section 7.2 shows that paying it amounts to laying a small protocol alongside every occurrence. Semantic substitution consolidates these coordination costs. A single interaction delivers the value to every occurrence at once, and the coordination is borne by the communication that was going to happen anyway. This is a richer form of compression than abstraction alone: variables with semantic substitution compress not only the description of *where* a value goes, but also the protocol by which it arrives *everywhere at once*.

Yoshida's blow-up can be read in these terms. In the recurrence $T(j{+}1) = 3T(j){-}1$ behind the count above, each level of nesting adds two kinds of atom to a body of n atoms. Rule (I) adds $n-1$ duplicators, which split the body at every parallel composition: that is routing, the part of the cost SKI would recognise. Rule (VI) puts every atom behind a synchroniser of its own, adding n more: that is coordination, one synchronisation per occurrence, which is exactly the per-occurrence structure of the deferred unquote. Composed down a chain of nested prefixes, both compound. The reflective encoding of [8] reaches $3d{+}1$ by quoting a continuation once and releasing it through a three-atom gate. The gate is a single synchronisation for the whole continuation, which is the combinator-level counterpart of installing received code by semantic substitution. Reflection does for the combinators what semantic substitution does for the term language: it consolidates coordination.

The comparison in this note also isolates the coordination component cleanly. $\mathsf{Sem}[D,V]$, $\mathsf{Unq}[D,V]$ and $\mathsf{ES}[D,V]$ all keep variables and binders, so routing is the same in each. Whatever a faithful translation from $\mathsf{Sem}[D,V]$ into one of the alternatives costs, it is the price of coordination alone, with abstraction held fixed.

Flat patterns sharpen the point. $\mathsf{Pat}[D,V]$ substitutes syntactically, as the alternatives do, and still delivers every occurrence in one interaction. So the special case in semantic substitution was never the source of the compression. What compresses coordination is binding

the components of the received value in the same interaction that delivers them.

11 Open questions

Open question 11.1 (Operational correspondence)**.** Translate $\mathsf{Sem}[D, V]$ into $\mathsf{Unq}[D, V]$ by reading $*y$ as $\&\, y$. Proposition 7.2 shows the translation is not adequate in general. Is it adequate up to weak bisimilarity on the fragment in which no drop occurs under quotation? The examples in Section 6 suggest it is, but this has not been proved.

Open question 11.2 (Relative pushouts with E2)**.** With E2 in $\equiv_S$, every output is congruent to a binary drop. Labels can then be taken in a normal form with binary drops on quoted names expanded. It remains to check that the reactive system for $\mathsf{Sem}[D, V]$ has the relative pushouts that the congruence theorem of [4] requires, and to state how semantic substitution interacts with the contexts involved.

Open question 11.3 (The explicit-substitution rules)**.** The rules for $\mathsf{ES}[D, V]$ here are a reconstruction. With the intended rules in hand, it should be checked which reading they fall under, and whether a middle reading exists that both keeps every resolution an interaction and restores atomicity.

Open question 11.4 (Which catalyst)**.** For $\mathsf{Unq}[D, V]^C$ the choice of catalyst matters: ambient or emitted, consumed or returned, silent or signalling. With a signalling catalyst emitted by the receiver, is there a systematic translation of $\mathsf{Sem}[D, V]$ into $\mathsf{Unq}[D, V]^C$ that reconstructs atomicity by protocol, and what does it cost, in steps and in the size of the bisimulations needed to prove it correct?

Open question 11.5 (Measuring the two compressions)**.** Can the split of Section 10 be made precise, as two measures on translations: one counting routing (duplicators, path construction) and one counting coordination (synchronisers, catalysed steps)? A natural first test is the translation of $\mathsf{Sem}[D, V]$ into $\mathsf{Unq}[D, V]^C$ with a signalling catalyst, where routing is unchanged and every cost of the translation should register as coordination; a second is to apportion the $\Theta(3^d)$ of Yoshida's elimination between the two measures.

Open question 11.6 (Lookahead)**.** Another way for a receiver to get a handle on p and d is lookahead,

$$\mathsf{for}(y \leftarrow x)\, P \mid x!(q)[k] \;\rightsquigarrow\; P\{\{\, s \mid q \rightsquigarrow^{j} s,\ j \le k,\ s \not\rightsquigarrow \,\}/y\},$$

with $k = *$ selecting the terminating states of the payload. It folds the payload's own computation into the COMM, so catalysed projections inside a payload would run within the communication step and regain atomicity. It remains to settle what substituting a set means (one reduct per state, or a single collective value), what a divergent payload contributes when the set is empty, how the j hidden steps are charged, and what to do about $k = *$, for which the set is not computable in general.

Open question 11.7 (Structure on D)**.** Nothing here uses structure on D. The quantum note [5] uses a group; other choices (costs, provenance tags, types) suggest themselves. The question is which properties of the core are stable under which choices, in particular whether erasure at the unary drop should be parametric in a map $D \to 1$ that the choice of D can refine.

Acknowledgement. The two alternatives, and the conversation that sharpened the argument, are due to Mike Stay. The note was drafted in dialogue with Claude (Anthropic).

References

[1] M. Abadi, L. Cardelli, P.-L. Curien and J.-J. Lévy. Explicit substitutions. *J. Funct. Program.* 1(4):375–416, 1991.

[2] H. B. Curry and R. Feys. *Combinatory Logic*, vol. I. North-Holland, 1958.

[3] P. Gardner and L. Wischik. Explicit fusions. *MFCS 2000*, LNCS 1893, 2000.

[4] J. J. Leifer and R. Milner. Deriving bisimulation congruences for reactive systems. *CONCUR 2000*, LNCS 1877, 2000.

[5] L. G. Meredith. Unitary in the equations, measurement in the reductions. Working note, F1R3FLY-io/publications, `fuzzyware/`, 2026.

[6] L. G. Meredith and M. Radestock. A reflective higher-order calculus. *ENTCS* 141(5):49–67, 2005.

[7] J. Parrow and B. Victor. The fusion calculus: expressiveness and symmetry in mobile processes. *LICS 1998.*

[8] F1R3FLY.io. Name-free combinators for the rho calculus. Research note, draft 3, F1R3FLY-io/publications, `rho-combinators/`, 2026.

[9] M. Stay and L. G. Meredith. Higher category models of the pi-calculus. *Higher-Dimensional Rewriting and Applications (HDRA)*, Warsaw, 2015. arXiv:1504.04311.

[10] N. Yoshida. Minimality and separation results on asynchronous mobile processes: representability theorems by concurrent combinators. *Theoret. Comput. Sci.* 274(1–2):231–276, 2002.